

Next: Enabling Continuous UTXO Payments with Direct Guarantees

Lu Hengrun

Abstract—Next is a native UTXO ledger enabling a recipient to verify a quorum-signed output certificate (TXCer) after one certification round and request a successor before public source execution, subject to successor admission and available capacity. If the successor executes first, the committee records a funded obligation against the direct issuer. Original signers can resubmit sources from publicly exposed certificates. A deadline-triggered decision debits the issuer’s reserve; late sources repay it, but permanently unexecuted sources leave the issuer bearing the CAL of publicly consumed outputs. Decision-authorized funding-reference repair then records historical reserve debits in designated inputs, supporting local exception review under the original block identity. It preserves owner authorizations and successor references without re-executing payments or delaying economic closure. For four-member organizations and a four-validator committee, each requiring three votes with at most one static Byzantine member, we prove unique consumption, coverage of outstanding certificate liability, and delay-neutral terminal CAL holdings for finite, acyclic, source-closed, conflict-free payment sets whose payments each succeed exactly once. On one host with synchronous member commits, median time to the last certified receipt in a 100-hop chain is 4.707s versus 64.479s with per-hop waiting. Targeted boundary and restart tests check capacity, economic closure, and historical reads.

Index Terms—Blockchain, certified payments, collateral, Byzantine fault tolerance, chameleon hash.

I. INTRODUCTION

In a UTXO ledger, each payment spends outputs of earlier transactions, so if recipients wait for public execution, every hop of a payment chain adds a confirmation delay. Suppose Alice pays Bob 100 CAL and Bob forwards it to Carol before Alice’s payment executes. If Bob’s payment executes first, the ledger consumes an output that does not yet exist, and the missing 100 CAL needs an identified, funded source, whether Alice’s transaction arrives later or never.

Waiting for public execution at every hop is the direct alternative, and our evaluation uses it as a control. Certificate-based systems separate recipient usability from public processing: FastPay and Zef support low-latency certified payments, and Sui Lutris combines broadcast and consensus paths for different kinds of state access [1]–[3]. Payment channels obtain rapid transfers from channel liquidity [4], [5]. Next asks how a native ledger can fund a certified payment whose source has not executed, with registered organizations rather than channels standing behind it.

Three difficulties are coupled. Certificates circulate before public registration, so accounting from public gaps misses existing commitments. A late source must not pay both the

guarantor that advanced the value and the recipient who already spent it. A client may hold a source and never submit it.

Our insight is that the party answerable for a missing source is already named by the certificate that travels with the output. Next charges the gap to that direct issuer instead of tracing ancestors, bounds every signed certificate’s liability by reserve-backed signing capacity, and closes an unresolved obligation by a public decision without changing stored history.

Economic closure leaves a separate record question: a compensated input still cites its missing output in the original block. Ledger auditing already uses saved execution evidence to revisit historical records [6]; Reparo also returns repaired transaction data through the old transaction hash [7]. We develop this workflow for compensation: an archival replica can export a revised funding reference at the original payment coordinates. An index supplies the same economic answer. The additional task is to authorize a replacement body while retaining the complete identity referenced by existing commits and successor headers [8], without executing the payment again.

The design has three complementary contributions.

C1: A layered architecture for one-round continuation.

Certification and public execution carry different duties: a certificate fixes outputs and assigns liability to its issuer, whereas public execution decides which payment the ledger accepts. Members of the consuming organization lock inputs and reserve capacity atomically before signing, and one parallel round yields a TXCer. The recipient verifies and records the output and can request a successor at once, carrying only certificates of its direct inputs, while replication and public submission run in the background. Successor approval remains subject to input, fee, and capacity checks; the stage-specific service contract is given in Section III-E.

C2: Direct responsibility with source decoupling.

Public execution atomically consumes a missing certified input and records an obligation against its issuer. The successor’s block publishes the source’s certificate, so the original signers can resubmit the source from stored approvals; after the deadline, a public decision debits the issuer’s reserve without threshold adaptation, and a late source repays it. Each approval reserves the full CAL output, change included, while it remains potentially certifiable, and an authenticated conflicting public spend may release the unused local reservation of a non-certified approval. For four-member organizations and a four-validator committee with at most one static Byzantine member each, we prove unique consumption and that signing capacity covers the outstanding CAL liability of every certificate, published or not.

For a finite, acyclic, conflict-free payment set that contains the sources of its certified inputs and executes each payment once, all orders of arrival and compensation end with the same CAL holdings and reserve balances. The accounting extends overlapping per-member budgets [9] to hidden certificate liability, dynamic compensation and repayment, and source closure.

C3: Decision-authorized funding-reference repair. A committed compensation decision closes an obligation economically; C3 reflects that decision in stored history. Restricted chameleon-hash commands write its reserve-debit reference into the designated input. Exact revisions and tasks authorize the replacement while owner authorizations, output and successor references, and the complete BlockID remain unchanged. Original commands are retained for replay, and representation writes no accounting state. This supports local exception review and re-export of a revised block under its existing identity, without making economic closure wait for historical repair.

On one M4 Max host with synchronous member commits and NoSync wallet stores, a 100-hop chain reaches its last certified receipt in a median 4.707 s, versus 64.479 s when waiting for public execution between hops. On the same current build, three fourteen-service P runs complete 21,048 payments; compensation and repayment finish before paused adaptation services resume. Separate experiments characterize behavior under injected network delays and single faults, and the local reading, storage, and maintenance costs of optional historical representation.

II. BACKGROUND AND RELATED WORK

Next combines certified payments that a recipient can reuse before public execution, reserve-backed responsibility for a source that has not executed, and authorized editing of stored history. We compare prior work along two questions: who answers for a missing source, and which payment and block identities survive once that answer is recorded.

Certificate-based payments and execution. FastPay uses Byzantine consistent broadcast for low-latency payments and, in its primary-ledger deployment, backs redemption with prepaid funds [1]. Its redeemability argument lets a valid certificate be confirmed before all incoming transfers are locally confirmed, provided the sender’s missing earlier outgoing certificates are supplied; redemption then reaches the primary funding pool. Zef combines disposable accounts with private, transferable coins, and ordinary spending uses current coin certificates and consumption evidence in place of the full ancestor history [2]. Sui Lutris pairs broadcast execution for owned objects with consensus for shared-object interactions [3].

Stingray supports concurrent transactions through Byzantine bounded counters and FastUnlock [9]. Its overlapping per-replica budgets (Sec. 4.2) and its treatment of budget exhaustion when no quorum certificate forms (Sec. 5.3) are the counting precedent for Next’s signing-capacity accounting, and we do not present them as new. Its commutative workloads differ from a payment chain in which each transaction consumes its predecessor’s output. For that chain, Next publicly executes a certified consumption before the source executes and registers

a funded obligation against the direct issuer. It adds responsibility for certificates that were never published, dynamic conservation of the Paid, Recovered, and Remaining quantities of each certificate, and closure of the source obligation by a public decision. The successor’s execution also publishes the source’s certificate, so the issuing organization’s own members can resubmit the source from the approvals they stored.

Channels and collateralized guarantees. Blitz replaces two-phase commitment in channel payments with pay-or-revoke [4], Donner develops virtual channels that address the Domino attack [5], and CryptoMaze provides atomic multipath channel payments with unlinkable partial payments [10]. All three operate through channel arrangements. Next runs on a native ledger with registered consumption routes and organization reserves, so counterparties need neither a bilateral channel nor a prefunded route between them.

Snappy protects merchants who accept fast on-chain payments through collateralized approvals, and reusable payment guarantees combine privacy with customer collateral that can be reused across defined time windows [11], [12]. Next keeps the user’s payment principal separate from additional organizational backing, and its compensation closes the missing source of an already executed child without a second principal credit to that child’s recipient. HyperService adjudicates cross-chain execution violations through a public Network Status Blockchain and an Insurance Smart Contract that financially reverses the effects of committed transactions [13]. Next draws the responsibility boundary within one native UTXO ledger: compensation settles a direct issuer obligation and leaves the principal effects of executed successors in place.

Controlled editing and historical consistency. Chameleon hashes permit authorized alternative messages under the same digest and underpin editable-blockchain designs [14]. SDR studies decentralized attribute-based edit authority and dynamic delegation [15], Escaping moves committee selection and voting outside the consensus layer to shorten redaction waiting [16], and Reconstructing Chameleon Hash develops stronger definitions of indistinguishability and collision resistance [17]. Next uses an established adaptation primitive, with the trapdoor held as threshold shares of an RSA exponent following Shoup’s practical threshold-signature approach [18]. The authority to apply a compatible replacement comes from a committed compensation decision and public payment-state checks, not from the primitive.

Reparo provides a publicly verifiable repair layer and can re-execute affected transactions while repairing state [7]. Its Ethereum implementation also maps an old transaction hash to repaired transaction data, supporting access through an existing record identifier. IA-CCF audits immutable ledger records against saved execution receipts [6]. These systems motivate review through established record references; Next specializes the revision authority to a committed compensation decision. Next fixes payment authorizations and outputs in advance: a public decision accounts for the compensation once, and representation commands replace only the permitted funding representation, without re-executing successors. The original commands, including the decisions, remain the economic replay history. RedactChain coordinates concurrent edits to

the same block and their preprocessing [19], and CDEdit uses reusable privilege tokens to cut repeated token and key generation and verification across multiple edits [20]. In Next, a batch shares the final block representation across the decided slots of one block, and each slot's reserve debit was already committed by its own decision, so the edit has no economic effect. The complete block identity combines the stable header hash with the original part-set header, and publicly committed logical revisions authorize exact replacement bodies. Physical installation can therefore advance directly to a later authorized revision without changing successor references or replay of the public command history.

III. SYSTEM MODEL AND OVERVIEW

Next lets a recipient spend a newly certified output before the transaction that creates it executes publicly. Four objects carry the design: authenticated payment rights, their public consumption, the responsibility for unresolved sources, and the later representation of that responsibility in stored history. Table I lists the notation.

A. Participants, Assets, and Assumptions

Next operates a native UTXO ledger. CAL carries payment value and FUEL pays processing fees. Users spend their own outputs, and organizations hold additional CAL in public reserve accounts. The fast path requires an output with a supported organization route; payment principal stays with its owner, and no bilateral channel is opened. Fees default to the user's final FUEL outputs; an authorized organization-funded mode also exists.

Each organization has four fixed members and a replaceable collector. Members validate requests independently and authorize payments, while the collector distributes requests, assembles certificates, and coordinates propagation without spending authority. A separate committee $\mathcal{C}$ of four equally weighted validators, with three votes required, orders public commands and maintains accounts, outputs, obligations, and compensation decisions.

Faults and cryptography. The adversary corrupts statically at most one member per organization and one committee validator throughout an execution; users and collectors behave arbitrarily. Honest participants share the network configuration, genesis, rules, and organization registry, and each route names one active configuration and one consumption-lock

domain. Signatures are unforgeable, canonical digests and original block commitments are binding, and messages may be delayed arbitrarily.

State. Honest signing state is atomic and never forgotten or rolled back. Before a member signs, its approval records the payment, the admission vector, and the direct input certificates of the request; the quorum certificate is not part of it. Public execution is deterministic and atomically committed, and followers apply authenticated consecutive blocks, updating state and cursor together. Compensation and representation read original blocks and their metadata, so correct committee replicas retain both at every height that holds an open obligation or a decided slot awaiting representation.

Adaptation and progress. Representation uses threshold adaptation with material generated and distributed by a trusted offline dealer, and a committed command must authorize any adaptation output. Progress needs responsive honest participants, available evidence, sufficient fees and resources, and fair delivery and scheduling. The experimental in-memory configuration keeps state atomic only while the process runs, and each run starts from fresh genesis.

Historical reads concern an honest replica that has verified and executed public prefix H , using one read transaction on its state V_H . Section IV-H specifies the interface semantics.

B. Outputs, Routes, and Certificates

An output carries an asset, an amount, an owner, and a consumption route. Its identifier x derives from the network, the creating transaction, and the output position, and spending is indexed by (x, i) with $i = 0$ for ordinary outputs. All inputs of a fast payment share one consumption organization, and the owner authorizes the transaction. New outputs may select another registered organization, so cross-organization continuation changes the route of the new output while the previous issuer's responsibility stays in place.

A TXCer pairs a summary with a quorum certificate (QC): three distinct member signatures over one fact c_T . The summary binds the network, transaction identity, issuer, configuration, epoch, rules, output commitments and amounts, and admission grants. Output bodies may be delivered separately, and the recipient checks commitment, position, and ownership before saving an output. The certificate authenticates the issuer's guarantee; Section III-E defines the corresponding service.

Each payment carries

$$\text{IntentID} = \text{Digest}(\text{INTENT}, \text{Network}, \text{Subject}, \text{Nonce}),$$

which names an operation of one payer on one network; it is not a nonce space shared among users. Members keep local intent records per organization, whereas public execution binds an intent to one TxID in the same atomic transition that executes the payment successfully. Two organizations can therefore certify, from independent inputs, two payments with the same subject and nonce and both obtain QCs, but at most one TxID executes publicly. This deduplicates transaction execution, not economic receipts from separately executed successors of conflicting QCs.

TABLE I
PRINCIPAL NOTATION.

Symbol	Meaning
$G, \mathcal{C}$	Guarantor organization and public committee
T, c_T	Payment and its certified summary fact
(x, i)	Output identifier and spend instance
g, B_g	Grant and cumulative authorization
a_T, p_T, Rec_T	Total CAL outputs, gross compensation, and recovered compensation
$\mathcal{O}_x, \mathcal{D}_x$	Direct obligation for a missing output and its immutable compensation decision
(H_b, P_b)	Stable block hash and original part-set header
ρ_ℓ, ρ_p	Committed logical and installed physical revisions

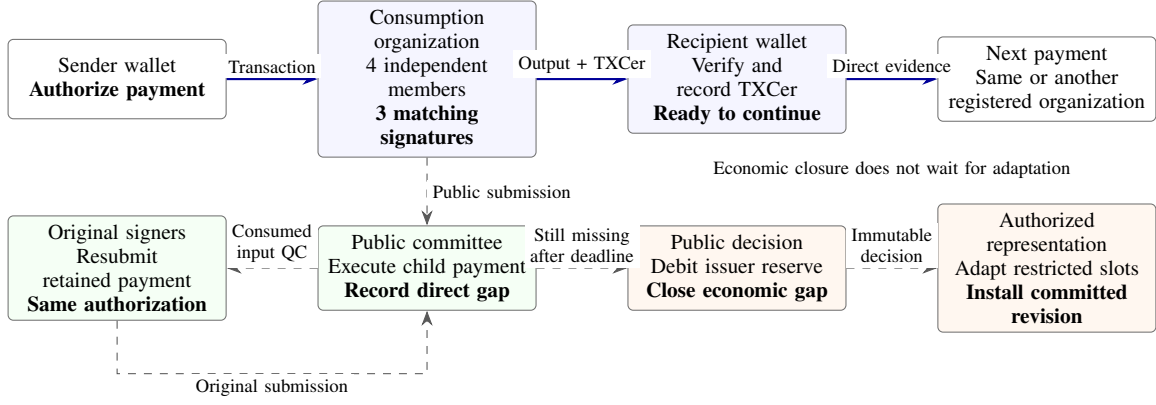


Fig. 1. One-round continuation and evidence-driven source resolution. Solid arrows form the recipient path; dashed arrows denote background work. Public successor evidence enables the original signers to resubmit retained authorized material. If the source remains unresolved after its deadline, the committee closes the gap independently of adaptation. A subsequent authorized representation preserves the payment and complete block identities.

Throughout, the *source* of an output (also its parent) is the payment that creates it, and its *consumer* (also its successor or child) is the payment that spends it. Child outputs are the outputs that the consumer creates. Source arrival means successful public execution of the source transaction, which the implementation’s *Settled* flag marks and which may coexist with nonzero *Pending*.

C. Three-Layer Design

Certification and continuation. A member reserves the input instances and applicable resources atomically before it signs. A CAL grant g authorizes B_g , and each member holds $\lfloor 2B_g/3 \rfloor$ signing capacity, partitioned among its local execution Workers. A payment reserves a_T , the sum of all its CAL outputs including change; these overlapping capacities constrain commitments but do not increase the backing funds. The collector returns a TXCer after three matching approvals in one parallel request–response round. Recipient verification and atomic local recording complete the fast path, while relay-material storage, *INSTALL* (replication of complete material within the organization), and public submission run in the background. Input locks outlive budget release.

Source-decoupled public execution. A public command carries the transaction, admission vector, organization authorization, and direct input certificates. The committee reconstructs the certified summary, verifies the authorization, and checks the current consumption state. For a certified input that is not yet publicly created, it atomically records the consumption, registers certificate-wide coverage against the issuer, creates $\mathcal{O}_x$ and its gap, and creates the child outputs; only a missing output that is actually consumed opens an obligation. Each public payment check uses direct certificates rather than fetching ancestor transaction bodies. Members restore signing capacity from authenticated blocks against their own recorded debits (Section IV-C). Genesis checks aggregate grants against each backing account, and top-ups debit unprotected accounts and raise balance and grant atomically; authorizations never decrease.

Resubmission, compensation, and representation. An obligation closes by source arrival or, after a deadline fixed by

public block time, by compensation from the issuer’s reserve. The block that executes the consumer publishes the certificate of its source, and members of the issuing organization that hold the approval rebuild the payment from retained material and resubmit it without new signatures. Once the deadline has passed, a public compensation decision can close an obligation that remains *Open*. It debits the reserve, closes the gap, and records an immutable decision without adaptation data; a source that arrives afterwards repays the compensating issuer within its own execution, and the decision stays recorded.

Representation commands then replace the funding reference of a decided input with the reserve reference. Restricted chameleon-hash adaptation preserves the fixed authorization, the output references, and the complete block identity (H_b, P_b) ; *RepairBatch* groups the decided slots of one historical block, and the commands write representation state only. Original commands, committed revisions ρ_ℓ , and installed revisions ρ_p stay separate, installation can advance directly to a later authorized version, and replay runs the original commands, which include the decisions. Successors need neither new signatures nor re-execution.

D. A Continuous-Payment Example and Design Goals

Alice pays Bob 100 CAL through G_A , creating output x routed to G_B . Bob verifies its TXCer and obtains a certificate for a payment to Carol. If Bob’s payment executes first, the committee consumes $(x, 0)$, records a 100-CAL obligation against G_A , and creates Bob’s outputs. Carol continues from Bob’s payment without Alice’s transaction body, and the block that carries Bob’s payment also carries the certificate of Alice’s payment, so members of G_A that approved it can resubmit it when Alice’s wallet does not.

Three outcomes follow. If Alice’s payment executes before compensation, it closes the obligation and instance 0 stays consumed. Once the deadline passes, a compensation decision can close an obligation that remains *Open* by debiting G_A by 100 CAL; Carol receives no second principal credit, Bob’s payment does not re-execute, and the stored block keeps the original funding reference until a representation command replaces it with the reserve reference. If Alice’s payment

TABLE II
SERVICE CONTRACT BY STAGE.

Trigger	Established state
Verified receipt	The issuer's QC binds the output and its owner. The recipient may request a successor, subject to its own admission checks. Receipt alone creates no public compensation obligation.
Public consumption of a missing certified output	Successful execution atomically consumes the input, registers its direct issuer's obligation, and creates the child outputs. Authorization, fees, consumption state, and Intent must pass the public checks.
Liability closure	Source execution fulfills the obligation; otherwise an included, valid post-deadline decision debits the issuer's CAL reserve. A later source repays it without a second credit to the recipient. The deadline alone does not guarantee completion.

executes after compensation, it consumes its own inputs under the same rules and repays G_A the 100 CAL; the decision stays recorded, Bob receives no second output, and $(x, 0)$ stays consumed. If Alice's payment never executes, G_A bears the loss.

E. Service Contract

Table II separates certified receipt, public consumption, and liability closure.

Terminal holders. A verified TXCer alone creates no redeemable public obligation. If its source does not execute and the holder cannot obtain a valid successor that executes publicly, the protocol offers no certificate-only redemption or unconditional exit; waiting does not create one.

Admission and stoppage. Any client may collect approvals, without acquiring spending or fee-sponsorship authority. Before a new approval, each member checks the Intent, input locks, fees, grants, and available capacity in every applicable Worker slice in one transaction, including a_T CAL for all outputs and change. Insufficient capacity rejects that approval without committing its changes or returning a signature. A retry of an existing, non-invalidated approval adds no reservation. The wallet records certified receipt only after verifying three matching signatures and the network, configuration, ownership, and output commitment. If no quorum approves, the request yields no new TXCer; earlier partial approvals remain reserved. Thus new certification can stop while the public reserve still has funds. Stoppage does not cancel existing certificates or unlock inputs.

Deployment and risk bearer. The implementation has neither Subject-level CAL admission nor a cross-collector unfinished-request bound. Fee-sponsorship policies and gateway concurrency limits do not supply these guarantees. Sustained progress depends on the workload conditions of Section V-F, rather than an enforced global admission limit. In open deployment, the direct issuer bears the principal of a publicly consumed certified output whose source never executes, including an

Intent loser. FUEL sponsorship does not transfer this CAL liability to the fee payer.

IV. PROTOCOL DESIGN

Next certifies fixed payment authorizations and outputs and lets their sources stay unresolved. Public execution consumes a missing certified input and records the issuer's direct obligation atomically, and it publishes the source's certificate so that the original signers can resubmit the source. A source still absent at its deadline closes through a public compensation decision without threshold adaptation, and representation commands later replace the funding reference under the existing commitments.

A. Identity, Certification, and Delivery

Let K_T be the encoded payment body and input claims, C_T the input commitments, and Σ_T the output summary:

$$\begin{aligned}
 K_T &= \text{Encode}(T.\text{Body}, T.\text{Claims}), \\
 \text{id}(T) &= H(\text{TX_V3}, K_T, C_T), \\
 c_T &= H(\text{DIRECT_CERT_V3}, \text{Encode}(\Sigma_T)).
 \end{aligned} \tag{1}$$

Owner signatures authenticate $\text{id}(T)$ and stay outside its computation, and each output identifier binds this identity, the network, and the output position. Funding references and openings are encoded separately, so a restricted replacement leaves the authorized claims and outputs unchanged. Initial submission requires `OriginalFunding` for each claimed input with a valid opening, and a later representation preserves body, claims, commitments, and owner signatures. Verification caches therefore key complete command bytes.

Under one Network and Subject, an honest wallet uses a fresh payment nonce for every new payment, across organizations and instances; a retry resubmits the same payment with its original authorization. A wallet must not change the nonce and reuse an old certificate, and two facts that share an intent are not merged into one guarantee. The experimental drivers derive nonces from input identities, which is not a general multi-device nonce allocation service.

A member checks ownership, routes, direct input certificates, value conservation, and the admission vector. One local transaction rechecks the input locks, final fee inputs, and the designated Worker's resource slices and records the approval with its exact debits; the approval keeps the transaction, admission vector, and direct input certificates of the request (Section IV-D). CAL reservation equals a_T including change even when every principal input is final, since each certified output may circulate before the payment executes. The member signs after the transaction commits, a retry reuses a non-invalidated approval, and releasing resource capacity keeps the consumption locks.

The collector sends one canonical request to four members and returns after verifying three signatures over the same c_T ; different valid subsets give the same fact. The recipient checks the TXCer against the output body, index, owner, and trusted configuration, records the coin atomically, and can construct a successor request at once, carrying its direct input certificates in place of the ancestor chain.

Relay material is saved in the background, and bounded early tasks let `INSTALL` and public submission proceed independently. `INSTALL` stores complete material at members and records certified consumption locally; it creates neither a new approval nor a budget debit. Public submission proceeds separately. Members schedule backup submission relative to their first `INSTALL`, and repeated `INSTALLs` cannot postpone it. HTTP acceptance leaves the outbox entry pending until authenticated public completion removes it. Backup submission requires a retained complete submission; Section IV-D adds a path from the approvals of the original signers.

The public `DirectSubmission` carries the transaction, admission vector, organization `SpendQC`, and direct input certificates. The committee reconstructs Σ_T and verifies the same authorization; omitting a separate new-output TXCer wrapper changes the envelope and keeps the requirement. Public verification binds each input certificate to its network, issuer, fact, and output index. It requires exactly one certificate-index entry per certified input, rejects duplicate `OutputIDs` and extraneous entries, and permits distinct outputs of one parent certificate. Coverage is registered once per parent fact. The admission vector holds one CAL entry, bound to the issuer.

B. Atomic Execution with Direct Obligations

Public execution separates certificate-wide coverage from an actual missing-input obligation. On the first missing use of a parent fact c , `register` reserves its entire certified CAL amount and creates a Promise for every output. The coverage record maintains

$$\text{Original}_c = \text{Discharged}_c + \text{Paid}_c + \text{Remaining}_c, \quad (2)$$

where Paid_c is a gross cumulative counter and $\text{Rec}_c \leq \text{Paid}_c$ counts the part repaid by source execution. Grant usage satisfies $\text{Reserved}_g + \text{Spent}_g \leq B_g$, where Spent_g sums $\text{Paid}_c - \text{Rec}_c$ over the certificates under g . Registration raises public grant usage and leaves the reserve account untouched; an effective compensation (Section IV-E) debits the account and raises Spent_g , and a repayment credits it and lowers Spent_g by the same amount. Only a missing output that is actually consumed opens a direct obligation and raises the public gap, so consuming one missing 40-CAL output of a 40/60 certificate reserves 100 CAL, opens a 40-CAL gap, and leaves the unused Promise covered.

Figure 2 gives the public transition. Its prestate includes earlier successful commands of the same block; static authorization may be cached, while consumption, balances, grants, and obligations are evaluated against the current state. All changes occur in a private overlay, and a failure discards the entire transition.

The intent check belongs to this evaluation: if the intent of T is already bound to a different TxID, the transition returns a conflict. The binding is written in the same atomic transition that executes a payment successfully, and no production path deletes or rebinds it, so resubmitting the same bytes cannot remove the conflict.

An obligation binds the consumed output, parent certificate fact, issuer, amount, consuming transaction, and historical

Require: Verified submission T , current state S , public time

- 1: If T is already settled, return no new effect
- 2: $O \leftarrow \text{Overlay}(S)$
- 3: Check intent, grants, and work; fund fee escrow in O
- 4: **for** each CAL input (x, i) **do**
- 5: Require (x, i) to be unconsumed
- 6: **if** a final creation for (x, i) exists **then**
- 7: Check its body and evidence against the claim
- 8: **else**
- 9: Require $i = 0$ and a matching direct TXCer c
- 10: Register all of c 's coverage if unseen
- 11: Create unique O_x ; add gap and Pending
- 12: **end if**
- 13: Record consumption of (x, i) in O
- 14: **end for**
- 15: Require equal total CAL input and output values
- 16: Record T settled; advance its fee stages
- 17: Resolve each output: fulfill an Open obligation; repay the issuer for a Repaired one and mark it Recovered; otherwise fulfill any registered Promise. Create final instance 0 unless repaid
- 18: Return all changes and results as one transition

Fig. 2. Public payment execution. Any failed check rejects all provisional changes. Output resolution follows Section IV-C.

input location, and public block time anchors its deadline. The consuming payment records its Pending input obligations and fee progress. User-funded fees consume authenticated final FUEL, return excess input value as change, and escrow the signed maximum, which covers registration, settlement, closure, and one compensation charge per certified input; unique stage and obligation identities prevent duplicate charging, and unused escrow returns to the payer when the input obligations close. The child outputs are final even while some inputs remain obligations, so a further transaction consumes them under the ordinary final-input checks.

C. Source Closure, Repayment, and Cumulative Reconciliation

A source transaction arrives through normal payment execution, including validation and consumption of its own inputs, and its outputs resolve within that transition according to their obligations. Arrival closes an Open obligation as Fulfilled: it fulfills the Promise, discharges the coverage, and lowers the consumer's Pending count. An output that no payment consumed fulfills any registered Promise and becomes final instance 0. A Repaired obligation means that a reserve debit funded the consumer, so arrival repays that debit instead of creating a second output: the amount returns to the issuer account that paid it, the obligation becomes Recovered, and the repayment counter and the grant's Spent change together, while the compensation decision stays as recorded. An already consumed instance 0 remains consumed in every branch; a repaid output has no final creation. The source's own inputs may open new obligations against their direct issuers within the same atomic transition.

Members reconstruct progress from authenticated consecutive blocks and results, and only an existing local approval supplies a debit to restore. For its CAL debit, member m

computes

$$t_{m,T} = \begin{cases} a_T - (\hat{p}_{m,T} - \widehat{\text{Rec}}_{m,T}), & \text{if settled,} \\ 0, & \text{otherwise,} \end{cases} \quad (3)$$

$$\delta_{m,T} = t_{m,T} - \text{Applied}_{m,T},$$

where “settled” means that the member has observed its own payment execute successfully, and $\hat{p}_{m,T}$ and $\widehat{\text{Rec}}_{m,T}$ are the compensation and repayment recorded locally for that approval’s outputs. On observing the execution of its payment, the member checks that the repayment in the authenticated result covers $\hat{p}_{m,T}$ and sets $\widehat{\text{Rec}}_{m,T} = \hat{p}_{m,T}$, since that execution repays every compensated output of T ; a member that approved T after following the compensation recorded none. After checking bounds and monotonicity, it moves $\delta_{m,T}$ from Reserved to Available in the original Worker and updates Applied. Pending inputs leave CAL release unaffected, since their issuers carry those liabilities, and work resources wait for fee closure. An unexecuted certified source keeps its full CAL amount, including change, reserved at each approving member; compensation alone does not release it. Spent FUEL is not restored as unspent fee. Repeated blocks and late material cannot repeat a credit. A wallet records a repaid output as a terminal observation without certificate, final creation, or balance.

An authorized reserve increase debits an external, unprotected CAL account, credits the reserve, and raises its grant atomically. Each member adds the difference between the new and old $\lfloor 2B_g/3 \rfloor$ shares while Reserved and its approvals stay in place, and aggregate backing checks cover all grants that share an account.

Superseded partial approvals. Equation (3) governs normal settlement and repayment. A separate local terminal transition applies when an authenticated, successfully applied public payment under the same organization configuration consumes a principal or user-funded fee input instance that the member had locked for a different fact. Before replacing the local spend candidate, the follower reloads the immutable approval and checks the exact output instance, the distinct fact and transaction, and the absence of observed execution, installed or queued certificate material, pending obligations, compensation, recovery, or any fee effect.

The member then returns, for every recorded local debit d , exactly $d \cdot \text{Cap} - \text{Applied}$ from Reserved to Available in the original resource slice and records the approval as invalidated by the first successful conflicting fact and height. It does not set Settled, close a fee escrow, modify the approval, clear other input locks, or change public state. The cumulative Applied vector makes the transition idempotent when several inputs of one approval conflict or a block is replayed. Retries and late INSTALL are rejected. Exposure of an invalidated fact as a purported certified source is treated as an accounting contradiction and stops the follower under the stated fault model. A timeout, INSTALL alone, or a global Intent conflict between disjoint inputs does not authorize this release.

D. Source Resubmission from Public Certificates

The block that executes a payment carries the quorum certificates authenticating its certified inputs and their creating payments. Each member follows the block in its authenticated block transaction. For every carried certificate of its own organization, it loads the approval indexed by the certificate’s fact, unless it has observed the source’s public execution or already holds an outbox entry for that fact, and checks that the stored transaction and admission vector reproduce the fact. It then assembles the stored transaction, the stored direct input certificates, and the exposed QC, and applies ordinary payment verification. On success it writes an outbox entry in the same atomic local update, staggering the first attempt by member index; the existing relay submits the entry after commit. Payments of other organizations trigger the same path, and a stored record that fails these checks is corrupted state that stops the follower.

The job resubmits the payment that the organization certified, so it needs no new owner or member signature, creates no approval, keeps input locks unchanged, and reserves no capacity. An existing outbox entry keeps its retry time, and execution of the source removes the entry. The path needs an original signer that holds the approval, follows the chain, and reaches the committee, and it needs that no global intent conflict prevents the source from executing; a source still absent at its deadline falls to compensation. It gives no fixed-time guarantee. Section V-B shows that at least two honest signers hold the material whenever an obligation is public.

E. Compensation Decisions

After its deadline, an Open obligation closes through a public CompensationDecision $\delta = (net, x, h, j, k)$: the network, the consumed output, and the height, transaction index, and input index of the consuming payment. The fixed-length canonical encoding holds neither a submitter signature nor adaptation data. Authority comes from the public obligation, and committed state fixes the target coordinates, so every submitter forms the same bytes for a given obligation. Committee workers scan the due index and submit due decisions without requesting adaptation shares.

Figure 3 gives the transition. The executor compares the request’s network, position, and input slot with the obligation before it reads any historical block, so a mismatching request is rejected without loading history. It then reads the canonical consuming payment at (h, j) , checks its identity, fact, input, and claimed amount, and requires OriginalFunding for x with a valid opening and a same-width reserve replacement; it computes no replacement opening and contacts no adapter. One overlay carries every change: the reserve debit, the Repaired status, Paid, Spent, the gap, the consumer’s Pending count, the compensation charge (from the consumer’s escrow to rewards), and, when no input obligation remains, the consumer’s fee closure and refund, together with the immutable decision record $\mathcal{D}_x$ (obligation snapshot and reserve-debit identity) and the pending-representation entry.

The result carries the single per-output effect, which names the parent and consumer facts, amount, input position, and

Require: Decision $\delta = (net, x, h, j, k)$, state S , execution height and block time

- 1: Require h to be older than the execution height
- 2: If $\mathcal{D}_x$ exists, return no effect when it equals δ ; otherwise reject
- 3: Require $\mathcal{O}_x$ Open with consumer position (h, j, k) and network net
- 4: Require the deadline of $\mathcal{O}_x$ to be reached by block time
- 5: Load the canonical consuming payment at (h, j) ; check its identity, fact, input, and claimed amount; require `OriginalFunding` for x with a valid opening and a same-width reserve reference
- 6: $O \leftarrow \text{Overlay}(S)$
- 7: In O : debit the issuer reserve; set $\mathcal{O}_x$ Repaired; move its Promise to Paid; update Spent, the gap, and the consumer's Pending count and fee stages
- 8: In O : record $\mathcal{D}_x$ and the pending-representation entry
- 9: Return the changes and the compensation result

Fig. 3. Compensation decision. The transition reads and requests no adaptation material. Any failed check rejects all provisional changes.

debit identity, together with any fee outputs. A repeated identical decision returns an unapplied result with an empty change set, also after the source's repayment, and a decision that differs from the record is rejected; once the source has executed, no new decision for its obligation commits. Members apply the effect from the authenticated result together with their cursor and locate original approvals for the Paid and Pending updates, and wallets apply each fee output once. The compensation credits no principal to the recipient or to an executed successor.

F. Representation Commands and Batches

A `RepairBatch` targets one historical block and holds 1–32 items in canonical order, each naming an output, a transaction index, an input index, and a new input opening; outputs and slots are unique. The batch fixes the base revision and the previous and replacement canonical-body digests, carries the part openings, and advances the logical revision from Base to Base + 1. A single `RepairInput` is the one-item form and carries the replaced transaction bytes. Both commands accept only outputs with a committed compensation decision, take amounts, issuers, and replacement references from the decision record, and write no accounting state.

Public execution repeats all checks against its current state. A changed base, an item without a decision, or a slot that no longer holds `OriginalFunding` rejects the entire batch, and exact successful replays are recognized before base validation. Since every command requires `OriginalFunding` in its slot, a different batch or the single-item entry cannot replace a slot twice, and command and revision identities stay specific to the chosen packaging. The authenticated result names the command and whether it applied; followers map it to an empty economic effect, and the economic effect of a compensation comes from the decision result alone. A repayment belongs to the source's own execution result.

G. Identity-Preserving Installation and Replay

Four identities are kept apart. The payment identity $\text{id}(T)$ fixes the authorization and commitments, and owner signatures

bind it; it is not the consensus transaction hash. The consensus $Tx.Hash$ of a well-formed UTXO4CH envelope is a SHA-256 over its header and fixed fields and checks no payment validity. $DataHash$ is the Merkle root over consensus transaction hashes, and an inclusion proof does not grant revision authority. The complete BlockID (H_b, P_b) keeps the block hash and the original part-set header, and execution and installation compare both. Part commitments are chameleon commitments bound to chain, key, height, and part index, and changed parts carry valid openings to their original commitments. Original consensus parts carry revision 0 and opening 1; original validation requires the correct height and these values, and the application verifies the original payment normally. A revised part cannot pass as a new original execution, and only a committed Task authorizes a rewrite. Replay uses the retained original commands and never executes a revised body as a new payment.

The public revision and Task authorize an exact new body and its part evidence, which an installer copies from one committed read view before it acquires the physical revision lock. For target revision r , a lower physical version is verified and installed, an equal version requires identical material, and a higher version covers the old task without rollback. The original block is retained, and parts and local version metadata are written atomically. Installation may skip intermediate physical versions and keeps every economic command. Queue progress stays global and consecutive: for entries A_1, B_1, A_2 , installing A_2 while handling A_1 leaves B_1 open. Logical execution reads committed Canonical revisions, so every installation schedule leaves the same economic history. A revision is *materialized* at a replica once its revised body has been physically installed there.

Replay executes retained original blocks in their committed order, including payments, compensation decisions, and representation commands, and reconstructs the application hashes of that sequence. Existing successors keep their authorizations, output references, and block identities. A compensation decision alone closes the obligation economically as an append-only public record. Representation adds an interface: at the consumer's original coordinates (h, j, k) , the stored block names the reserve debit for the designated input and still verifies against the original BlockID. A reader still reads the permanent decision and the obligation status for the economic state, and the revised field does not replace them. Batching shares representation work within a block and direct installation coalesces historical writes, and neither adds a round to normal certification. Section VI-C reports the functional distinction and the read, storage, and maintenance tradeoffs against an optimized decision index.

H. Reading Funding at Original Coordinates

A read at (h, j, k) returns $(H, h, j, k, F, r, d, s_H)$, assembled from existing functions inside one read transaction on V_H . Here F is the funding representation, r its revision, d the decision or an absent marker, and s_H the obligation state. This logical tuple describes local composition, not a deployed network interface. The index j counts the original transaction

list and is not renumbered after filtering maintenance commands or failed transactions. The logical body comes from the committed revision in V_H and falls back to the original block only when no revision exists, so physical installation progress, or a block store that already holds a later version during replay, does not change the answer. A revision that exists but cannot be decoded is an error and never falls back to the original. Missing history is reported as unavailable and is not read as the absence of compensation; the contract assumes that the relevant history is not pruned. In a complete, verified, unpruned V_H , a missing decision means that none exists as of H .

Four state dimensions. These dimensions describe different objects and advance separately.

Dimension	Meaning
Public payment	Settled records successful execution. Newly created outputs are final even when Pending is nonzero; outputs with Repaired obligations repay the reserve rather than being recreated.
Direct obligation	Open→Fulfilled, or Open→Repaired→Recovered. Repaired denotes compensation; Recovered denotes subsequent repayment. The compensation decision persists.
Logical representation	The revision in V_H determines the slot. OriginalFunding does not imply absence of compensation. ReserveFunding records a historical advance and remains after repayment.
Physical installation	ρ_p records the installed block-store version. Installation progress does not determine the logical answer at H or change economic state.

The reader therefore consults the permanent decision and obligation state together with the funding representation. A pending-representation entry is removable work state, not durable evidence that compensation occurred. None of these representation or installation changes credits a second principal amount to the consumer.

Use: exception review and historical re-export. A local archival service revisits the input of an executed payment to report its historical reserve advance, authorizing decision, and repayment status. For example, Bob’s payment to Carol consumes Alice’s missing output at (h, j, k) . A subsequent decision debits Alice’s issuer. The original input accurately records the object accepted at execution; the revised funding slot records the later advance.

The service has verified and executed prefix H . From the same V_H , it obtains the authorized body, exact Revision and Task, permanent decision, and current obligation state. It reconstructs the block and parts with the authorized openings, checks the complete BlockID against the saved original commit, and exports the original and revised bodies with their decision and revision context. The commit checks compatibility with the existing block identity; executed commands and the exact Task authorize the replacement bytes. Because votes cover both the header hash and part-set header, preserving the header alone is insufficient. If Alice’s source later repays the issuer, the debit reference remains a record of the historical advance while the obligation becomes Recovered.

C3 therefore has a correctness target distinct from the amount settled by C2: even a correct reserve debit must not authorize a body that attributes the input to another

issuer’s debit. The repair must preserve both the permitted funding reference and the fixed payment and block identities; decision-derived checks reject such a candidate. Our prototype implements this Next-aware local workflow (Table IV); it requires the executed prefix, rather than treating an exported body as a standalone remote proof.

An index or ordinary materialized view answers the same economic questions while keeping the original body. A separately authenticated revised copy could also retain an old-ID lookup, as illustrated by Reparo [7]. C3 instead makes the authorized revised body itself satisfy the original complete BlockID. This is its record-level service, not an extra condition for payment finality. Economic closure proceeds with the representation service stopped; the current build still uses CH transaction and part encodings.

Cryptographic instantiation. The implementation uses RSA chameleon hashing with a 2048-bit modulus, $e = 65537$, and context-bound SHAKE256 full-domain hashing into $\mathbb{Z}_N^*$. A trusted dealer generates a three-of-four threshold RSA key using CIRCL v1.6.5 [21], following Shoup’s threshold construction [18]. Public adaptation ratios do not convey edit authority; committed decisions, exact Tasks, and complete-identity guards do. The supplement specifies the encoding, parameter assumptions, and raw-group adaptation interface.

V. SECURITY ANALYSIS

We analyze finite executions of the protocol of Section IV under the model of Section III-A. The arguments reason over the guards of the transition functions and use, besides the fault and state assumptions, the verification rules of Section IV-A: input certificates bind network, organization, fact, and claimed output and correspond one-to-one with certificate inputs, and each admission vector holds one CAL entry bound to the issuer. A representation takes effect only through a committed command whose guards require an existing compensation decision, derive the slot and reserve reference, and verify every non-target byte and the complete block identity, so adaptation output is a candidate whose availability governs representation progress alone. The consensus argument specializes the Tendermint locking interface [22] to complete block identities.

A. Original Execution and Certified Consumption

A *polka* is a same-round quorum of prevotes for a complete block identity or nil.

Lemma 1 (Original-execution agreement): Under the locking and validity rules of the consensus interface, one original block body is admitted per committed height, including its application context.

Proof: With original opening 1, a part leaf hashes its context-bound message representative, so binding and the Merkle proof fix its bytes under the original part-set header. Candidate bodies are rebuilt from parts or paired locally with parts, Locked and Valid transfers preserve the pairing, and proposal, proof-of-lock (POL), lock, and commit comparisons use (H_b, P_b) ; a change of download target clears the old body, and the round and step guards allow one precommit per round.

Conflicting same-round commits need an honest double precommit. Take a commit of v in round r with its two honest signers, and the smallest later round s with a conflicting polka (nil polkas that authorize unlocking included) at its earliest certificate formation. The quorum meets the two signers, and the vote of that signer needs an earlier unlocking justification, which contradicts the choice of s or of the formation event. Votes for a locked value trace through earlier lock rounds to proposal validation, and Finalize follows a commit that matches the complete block identity. Synchronization and replay preserve body, height, and time, and the supplement expands the argument. ■

Lemma 2 (Authenticated, unique consumption): A TXCER fixes its outputs and issuer. No two valid certificates authorize conflicting consumption of the same (x, i) , and public execution consumes that instance at most once.

Proof: Binding fixes the certified summary and the owner-authorized transaction, and the fixed route sends competing requests to one organization. Two three-member quorums intersect in at least two members, so an honest member lies in both. Its atomic input lock precedes signing, so it approves only one of two conflicts, and budget release keeps the lock. Final-input and certificate-input execution consult the same spend key. Replaying a fact adds neither a consumption nor a fee charge. A repaid output has no final creation and its instance 0 stays consumed, so a later claim on (x, i) fails. ■

Lemma 3 (Superseded partial approval): Fix one four-member organization with three signatures required and at most one Byzantine member. Let an honest member have approved fact c and locked a principal (final or certified) or user-funded fee input instance ξ . If a successfully applied public payment under the same organization configuration carries a valid QC for a different fact c' and a different transaction identity and consumes ξ , then no three matching valid member signatures for c can have existed. The honest member may therefore restore the residual local debits of c while permanently refusing to sign or install c . This conclusion does not apply to two QC-bearing facts with disjoint inputs that conflict only through their global Intent.

Proof: The public execution of c' has passed the current organization's authorization checks and therefore carries a valid three-member QC. Assume for contradiction that c also has a valid three-member QC. The two signer sets intersect in at least two members. Since at most one member is Byzantine, an honest member belongs to the intersection. That member would have signed two different facts that consume the same instance ξ , although honest members atomically lock principal and user-funded fee instances before signing and reject a different candidate for a locked instance. This contradicts the approval rule.

Thus no three matching signatures for c can have existed. In the notation introduced in Section V-C, $c \notin C_g$. Restoring one member's residual debits for c cannot remove a capacity witness for any term of Q_g . The successful public consumption of ξ , the immutable approval, and the invalidated progress record continue to reject later use of c ; only its unused local resource reservation is restored. ■

B. Source Resubmission and Compensation Decisions

For an output x , $\mathcal{O}_x$ is its direct obligation, with status Open, Fulfilled, Repaired, or Recovered, and $\mathcal{D}_x$ is its immutable compensation decision. Repaired means economically compensated, whatever the state of the stored bytes. The decision is written inside the transition from Open to Repaired and is never deleted, so $\mathcal{D}_x$ exists exactly when $\mathcal{O}_x$ is Repaired or Recovered. The set $\mathcal{R}$ holds the representation state (committed revisions, head command, tasks, batch references, queue, pending-representation index), and the projection π_{econ} retains public assets, consumption, coverage, obligations, grant usage, and fee states while omitting $\mathcal{D}$ and $\mathcal{R}$.

Lemma 4 (Resubmission preserves authorization): Let an honest member create a resubmission job for a fact c . The job submits the payment, admission vector, and direct input certificates that the member stored before it signed c , with the public QC of c , and public verification and execution treat it as any submission of that payment. It needs no new owner or member signature, creates no approval, keeps input locks unchanged, and reserves no signing capacity. A member holds at most one job per fact, and execution of the payment removes it. Once a direct obligation on an output of c is public, at least two honest signers of c retain the stored material.

Proof: The member indexes its approval by the fact in the public QC, and the stored payment and admission vector must reproduce that fact. It combines this material and its stored direct input certificates with the exposed QC and applies ordinary payment verification before it creates an outbox entry. Fact binding and quorum verification exclude any other payment core, and the quorum signatures fix the content of the certificate that the exposing party chose to carry. The transition writes the outbox alone, so the approval, input locks, and grant reservation stay as they were, and the existing outbox and observed-execution marks suppress duplicates. A stored record that fails these checks lies outside the fault model.

An obligation arises only from an executed payment that consumes a certified missing input, and public verification requires that input's certificate, which is therefore public. Its three distinct signers include at least two honest members, each of which stored the approved material before it returned a signature. The approvals hold no quorum certificate, which arrives with the block, so a client that withholds the certificate leaves the material for resubmission intact. Execution further needs a continuing honest follower, fair delivery, and a source payment that meets the ordinary execution rules, including the global intent check. ■

Lemma 5 (Certificates bound liability, not execution): Let $T_1 \neq T_2$ be payments with the same Network, Subject, and Nonce, spending disjoint inputs routed to organizations G_1 and G_2 , each valid under its own organization's approval rules. Both payments can obtain QCs, and in every public order at most one executes. A malicious user can cause this, and resubmitting either payment does not remove the conflict. Payments of different Subjects with the same nonce do not conflict.

Proof: Each organization holds its own local intent record and the approvals concern disjoint inputs, so neither organiza-

tion observes a conflict, and the owner's authorization covers each payment. At public execution, the evaluation returns a conflict when the intent is already bound to a different TxID. The binding is written in the transition that executes a payment successfully, and no production path deletes or rebinds it, so the other payment stays unexecuted whatever its resubmission. A different Subject yields a different intent. ■

If T_2 cannot execute but a consumer of its certified output does, the consumer registers an obligation against G_2 . Compensation debits that issuer without making T_2 executable. Its approving members retain $r_{m,c} = a_c$, and its liability remains $w_c = a_c$. The original inputs stay locked, but a user controlling the consumer's output may use that new value in a later request. Repetition requires fresh paired Intents, usable routes, certification capacity, and fees; the final descendant must also retain a usable route. This reuses descendant value rather than unlocking the rejected source inputs. In user-funded mode, the rejected source retains locks on its complete FUEL inputs, not merely its signed escrow maximum. Theorem 3 excludes this case because not every payment succeeds. Terminating repeated submissions of the conflicting source would not authorize release of its certified liability.

Lemma 6 (Compensation takes effect at most once): Each direct obligation incurs at most one reserve debit and its associated economic effects. A decision for x with an existing record $\mathcal{D}_x$ has no effect if it equals the record, including after repayment, and is rejected otherwise. A decision without a record is rejected once $\mathcal{O}_x$ is no longer Open.

Proof: Status changes only along Open→Fulfilled, Open→Repaired (by a decision alone), and Repaired→Recovered (inside the successful execution of the source). A decision first checks $\mathcal{D}_x$: an exact match returns an unapplied result with an empty change set, and a mismatch is rejected. Without a record, the transition requires an Open obligation, a deadline reached by block time, and a matching historical position, amount, and funding commitment, all re-derived from committed state. It then debits the issuer reserve, closes the gap, updates fees and coverage, and records $\mathcal{D}_x$ in one atomic step. The status machine is acyclic, so Open is left once and at most one decision takes effect.

If the source executes first, the obligation leaves Open and no decision for it commits. If compensation comes first, the source's execution returns the funds to the issuer through the single Repaired→Recovered transition, keeping $\mathcal{D}_x$ and creating no user output. Induction over the public command order gives at-most-once compensation and repayment. ■

C. Covering Public and Hidden Commitments

Every certified fact carries one CAL entry bound to the issuer, because at least two honest signers computed the vector through the admission rules before they signed and the fact binds the summary. For a complete CAL grant g , the cumulative authorization B_g gives each member $s(B_g) = \lfloor 2B_g/3 \rfloor$ capacity. Let C_g contain every distinct fact whose three signatures have existed by the current prefix, including unassembled and hidden certificates and settled facts. The

unique CAL allocation binds each fact to one configuration, ResourceKey, and Grant ID. A fact c has output cap a_c , the sum of its CAL outputs. For $c \in C_g$, let p_c be the cumulative compensation paid for those outputs, Rec_c the part repaid by source execution, and z_c indicate that the payment of c has executed successfully. The *net compensation* is $\bar{p}_c = p_c - \text{Rec}_c$. Define

$$w_c = \begin{cases} a_c, & z_c = 0, \\ \bar{p}_c, & z_c = 1, \end{cases} \quad Q_g = \sum_{c \in C_g} w_c. \quad (4)$$

Lemma 7 (Source repayment): A compensated output leaves the Repaired state only inside the successful execution of its source, in the same transition that sets $z_c = 1$, and the repayment is credited to the account that was debited. Hence $0 \leq \text{Rec}_c \leq p_c \leq a_c$, $\text{Rec}_c = 0$ while $z_c = 0$, and $\text{Rec}_c = p_c$ once $z_c = 1$. In particular $\bar{p}_c = p_c$ while the source is pending and $\bar{p}_c = 0$ afterwards, so no net compensation survives the execution of the source.

Proof: Each registered promise leaves Open at most once, and by Lemma 6 compensation applies at most once to an output and only while its obligation is Open, so $p_c \leq a_c$. The only transition that leaves Repaired is the output-resolution step of a payment whose fact and issuer equal the certificate fact and issuer of the obligation. It runs in the overlay that records the payment as settled, so z_c and the repayment change together. The step resolves every output of the source that has an obligation, turning Open into Fulfilled and Repaired into Recovered, and afterwards every promise of c is closed, so no new compensation arises and a repeated submission is a no-op. Compensation debits and repayment credits use the issuer account stored in the obligation, which is the account of the single CAL allocation of c , and the repayment leaves $\mathcal{D}_x$ untouched. ■

For any local approval, let $r_{m,c} = a_c - \text{Applied}_{m,c}$, where Applied is its cumulative CAL restoration.

Lemma 8 (Residual capacity): For every fact $c \in C_g$ and every honest member m that approved c , at every prefix, $r_{m,c} \geq w_c$.

Proof: For $c \in C_g$, Lemma 3 excludes invalidation by an honest signer: the triggering conflict would imply $c \notin C_g$. Until m observes the successful execution of the payment of c , its CAL restoration for c is zero, so $r_{m,c} = a_c \geq w_c$ whatever the compensation state. After it observes that execution, m checks that the reported repayment covers its recorded compensation and sets its recorded repayment equal to it. The restoration target is a_c minus a recorded net compensation of zero, so $r_{m,c} = 0$, and $w_c = \bar{p}_c = 0$ by Lemma 7. A member that approved after following a compensation recorded none and restores only its own debit. ■

Theorem 1 (Potential liabilities are funded): For every CAL grant, $Q_g \leq B_g$. For each reserve account x , its total open gap Gap_x does not exceed its balance A_x .

Proof: Let $\mathcal{A}_{m,g}$ contain all successful approvals under g , including those without a QC (invalidated approvals contribute zero residual), and $L_{m,g} = \sum_{c \in \mathcal{A}_{m,g}} r_{m,c}$ across Workers. Approval, ordinary restoration, and superseding invalidation move capacity between Available and Reserved, and top-ups

add the difference in member shares, so $L_{m,g} \leq s(B_{m,g}) \leq s(B_g)$ for the locally observed authorization $B_{m,g}$; grants only increase. Fix any three honest members H_g of the organization. A QC has three distinct signers and at most one member lies outside H_g , so the set Sig_c of members of H_g that signed c has at least two elements. Honest members sign only after approving, so Lemma 8 gives $r_{m,c} \geq w_c$ for $m \in \text{Sig}_c$, and

$$\begin{aligned} 2Q_g &\leq \sum_{m \in H_g} \sum_{\substack{c \in C_g \\ m \in \text{Sig}_c}} r_{m,c} \\ &\leq \sum_{m \in H_g} L_{m,g} \leq 3[2B_g/3] \leq 2B_g. \end{aligned} \quad (5)$$

Let R_c and d_c be the registered remaining and discharged coverage, with $p_c = d_c = R_c = 0$ before registration. Registered records satisfy $a_c = p_c + d_c + R_c$, and $z_c = 1$ implies $R_c = 0$. Hence $R_c \leq w_c - \bar{p}_c$: while $z_c = 0$ we have $\text{Rec}_c = 0$, so $R_c \leq a_c - p_c = w_c - \bar{p}_c$, and once $z_c = 1$ both sides vanish. Define $S_g = \sum \bar{p}_c$, $V_g = \sum R_c$, and $E_g = Q_g - S_g$, with sums over C_g . Registration, compensation, repayment, and discharge maintain public $\text{Usage.Spent} = S_g$ and $\text{Usage.Reserved} = V_g$.

Let $\mathcal{G}_x$ contain all grants backed by x ; their certified CAL account is the debited issuer account. Genesis establishes $\sum_{g \in \mathcal{G}_x} (B_g - S_g) \leq A_x$. External, non-protected top-ups raise both sides equally. Compensation raises S_g and lowers A_x by the same amount, repayment lowers S_g and raises A_x by the same amount (Lemma 7), and no other protocol operation withdraws CAL reserves. Each Open obligation corresponds to a distinct remaining promise, so

$$\begin{aligned} \text{Gap}_x &\leq \sum_{g \in \mathcal{G}_x} V_g \leq \sum_{g \in \mathcal{G}_x} E_g \\ &\leq \sum_{g \in \mathcal{G}_x} (B_g - S_g) \leq A_x. \end{aligned} \quad (6)$$

Gross Paid and its recovery are each counted once, and E_g is a risk bound rather than available member capacity. ■

A partial approval remains charged until either its own payment executes or authenticated public history proves that one of its locked instances was consumed by a different successfully authorized fact under the same configuration. Time alone never releases it. A disjoint-input payment that loses a global Intent conflict may still have a valid QC and issuer liability, and therefore retains its input locks and signing reservation.

CAL admission consequence. At the execution prestate, including earlier successful block commands, let N_g be the distinct, valid, unregistered parent facts under g that matching, unconsumed missing inputs require. They have $w_c = a_c$ and $p_c = R_c = 0$. Every registered fact satisfies $R_c + \bar{p}_c \leq w_c$, and the facts are disjoint, so

$$V_g + S_g + \sum_{c \in N_g} a_c \leq Q_g \leq B_g. \quad (7)$$

Repeated uses of one certificate count once, and sequential registration sees earlier reservations, so every partial sum obeys the bound. First registration therefore cannot fail for CAL authorization capacity, even when it precedes a repayment in the

same transition, since a repayment only lowers S_g . Likewise an Open amount $a \leq \text{Gap}_x \leq A_x$ has sufficient reserve principal. Other resource and fee checks remain separate.

D. Asset Conservation and Delay Neutrality

Theorem 2 (CAL conservation): Let U be the total value of final, unspent CAL instances, A the total CAL account balance, and D the total value of Open gaps. Every execution prefix satisfies

$$U + A - D = C_0, \quad (8)$$

where C_0 includes genesis outputs and all accounts, including top-up sources.

Proof: A payment consumes final value F and missing value M and produces $F + M$. Let J and R be the total values of its arriving outputs whose obligations were Open and compensated, respectively. The former outputs were already consumed, so they close gaps and add no unspent value; the latter receive no creation record. Hence $\Delta U = M - J - R$, $\Delta D = M - J$, and the repayment gives $\Delta A = R$, so $\Delta(U + A - D) = 0$. A compensation decision of amount a gives $\Delta A = \Delta D = -a$ and $\Delta U = 0$. Resubmission changes no public state, and by Lemma 10 representation commands and installation leave U , A , and D unchanged. Top-ups transfer between included accounts, and other transitions preserve the projection. ■

FUEL follows separate accounting: each escrow maximum equals Held plus cumulative Rewards, Burned, and Refunded, and the global conserved sum counts unspent FUEL, ordinary accounts, current Held, reward balances, and cumulative burn. Unique stage and obligation identities transfer value once, for user-funded and organization-funded fees alike.

Conservation fixes totals, and the next result fixes the final holders.

Theorem 3 (CAL delay neutrality): Let $\mathcal{T}$ be a finite set of payments such that (i) no two consume the same instance or fee input, (ii) the relation “consumes an output created by” is acyclic, and (iii) every certificate input of a payment in $\mathcal{T}$ certifies an output of a payment in $\mathcal{T}$. Let $\mathcal{U}$ be a finite set of authorized reserve increases. Start from a clean genesis state with no coverage, promise, or obligation records and zero CAL Reserved and Spent. Let ρ_0 execute $\mathcal{T}$ in an order consistent with (ii), applying $\mathcal{U}$, and let ρ be any execution consisting of the payments of $\mathcal{T}$, the increases of $\mathcal{U}$, and compensation decisions of due Open obligations, in any order, optionally interleaved with admissible representation commands. Suppose every payment and increase succeeds exactly once in both executions. Then at the end of ρ and of ρ_0 :

- 1) the final, unspent CAL instances coincide, with the same owners and amounts;
- 2) every reserve account holds the same CAL balance;
- 3) every CAL grant has the same authorization and $\text{Reserved} = \text{Spent} = 0$, every obligation is Fulfilled or Recovered, and $\text{Paid}_c = \text{Rec}_c$ for every certificate.

Fees and FUEL are not compared.

Success here includes all applicable authorization, input, Intent, resource, and fee checks. The CAL coverage bound

does not imply that every QC-bearing payment meets these independent execution conditions.

Proof: Holders. By (i), both executions consume the same instances. In ρ , an output consumed before its source executes receives an obligation, and when the source executes the output is either Fulfilled, with a creation record that is already consumed, or Recovered, with no creation record; in both cases it is consumed. Outputs that no payment of $\mathcal{T}$ consumes are created as final and unspent in both executions. The unspent instances are therefore the genesis outputs and the outputs of $\mathcal{T}$ that $\mathcal{T}$ leaves unconsumed.

Reserves. For a reserve account x , let $\hat{A}_x$ be its balance plus the amounts of Repaired obligations issued by x . Compensation lowers A_x and raises the Repaired total by the same amount, repayment reverses this on the same account (Lemma 7), and creating or fulfilling an obligation changes neither term. As in Theorem 1, only top-ups, compensation, and repayment change reserve balances, so $\hat{A}_x = A_x(0) + \tau_x$ at every prefix, where τ_x is the total of increases credited to x so far. An obligation exists only for an output consumed before its source executes, and by (iii) that source executes in ρ and leaves the obligation Fulfilled or Recovered. At the end no obligation is Open or Repaired, so $A_x = A_x(0) + \tau_x$, which is also the value in ρ_0 .

Usage. Every registered certificate is used by a missing input, so by (iii) its source executes in ρ . That execution closes all of its promises, so Remaining = 0, and Lemma 7 gives Paid_c = Rec_c. Hence Reserved = $\sum$ Remaining = 0 and Spent = $\sum$ (Paid_c − Rec_c) = 0, and authorizations change only through $\mathcal{U}$.

Representation. By Lemma 10, an interleaved representation command leaves every compared quantity and the result of a later payment or decision at the same height and time unchanged. ■

Intermediate states differ: a compensated source's issuer holds less until the source executes, which Theorem 1 covers, and the compared projection omits the decision records and representation state that exist only in ρ . A source that never executes leaves its compensated amount as a net loss of its issuer, and incentives lie outside the theorem.

E. Authorized Representation and Independence

Lemma 9 (Decision-gated representation authority): Every effective representation item refers to an existing compensation decision, including one whose obligation has since been repaid, and to a canonical slot that still holds OriginalFunding. A successful representation changes no transaction bytes outside its authorized funding slots, even when adaptation outputs are adversarial.

Proof: Let N be the RSA modulus, e its public exponent, and r an opening in $\mathbb{Z}_N^*$. The domain-separated representative FDH(ctx, m) binds the message and its repair context. The relation

$$C = \text{FDH}(ctx, m)r^e = \text{FDH}(ctx, m')r'^e \pmod{N} \quad (9)$$

gives compatibility and carries no fresh approval, and a published opening ratio can be reused for the same message

relation. Public guards read the decision record, which holds the obligation snapshot and the reserve-debit identity, and derive the slot and the replacement reference from it. They check the current base and body digests and verify all non-target bytes and the complete block commitments. Reading the snapshot keeps a decided slot representable after repayment. Invalid candidates leave no changes, exact successful replays are no-ops, and a repackaged command meets a slot that no longer holds OriginalFunding or an existing task. The decision committed the economic effect, so the command supplies no amount, issuer, or account change, and the adapter writes no ledger state; the argument rests on the verified output alone. ■

Lemma 10 (Representation is economically inert): For every successful single or batch representation command r and state S , $\pi_{\text{econ}}(r(S)) = \pi_{\text{econ}}(S)$. A payment or decision executed at the same height and time yields the same result in $r(S)$ as in S , and the projections of the two successor states agree.

Proof: A representation command checks the decision, the canonical target, the base revision, the preceding digest, and the permitted replacement. Its write set holds revisions, the head command, task and batch-reference records, the queue, and pending-index removals; it skips the accounting transition and writes no account, obligation, coverage, usage, spend, or fee record. Followers map its result to an empty economic effect. The projection is therefore unchanged for one step and, by induction, for any sequence of such steps.

A decision for an output y reads the canonical body only through the transaction identity, the authorization fact, the inputs, claims, and amounts, the encoded length, and the slot of y . If $\mathcal{D}_y$ exists, the decision is a no-op that reads no history. Otherwise the slot of y still holds OriginalFunding, since replacement requires $\mathcal{D}_y$, and replacing other fixed-width slots leaves the remaining fields unchanged. Payment execution never reads the canonical body. ■

Lemma 11 (Identity preservation and installation independence): An effective representation preserves the owner authorizations, transaction and output identities, claims, successor references, and the complete original BlockID (H_b, P_b). For the same public history, legal physical installation schedules have identical economic projections. Moreover, if the chameleon relation has a unique valid opening for each fixed message and commitment, then for a fixed original body and a fixed set of decided slots the final body and part openings do not depend on the grouping or order of representation commands. Revision numbers, command histories, and application hashes may differ.

Proof: The executor builds the canonical body by changing only the declared funding slots and their openings. Transaction identity, claims, and input commitments stay fixed by the identity encoding of Eq. (1), and replacements keep the encoded width. The executor verifies the new openings, recomputes the block hash and part-set header of the revised body, and accepts only equality with the original BlockID. Base and Previous serialize competing canonical revisions, and the committed Task binds the permitted bytes. Successors reference transaction and output identities, so they need neither re-signing nor re-execution.

Canonical reads the committed logical revision ρ_ℓ , and an exact Task authorizes body and parts at revision $r \leq \rho_\ell$. Installation advances if $\rho_p < r$, checks exact equality if $\rho_p = r$, and treats $\rho_p > r$ as covered without rollback. It writes no economic state and reads only the committed revision and task records, and queue progress crosses only completed entries. For legal schedules μ_1, μ_2 over one public prefix this gives

$$\pi_{\text{econ}}(S_{\mu_1}) = \pi_{\text{econ}}(S_{\mu_2}). \quad (10)$$

The convergence statement follows from the uniqueness of each slot and part opening and the independence of part contexts from the revision counter. The supplement gives this argument and the comparison of follower updates and histories. ■

Reader scope. These lemmas concern a replica whose own state ran the guards, so the reader of Section IV-H trusts local state that executed them. A block that carries a `RepairBatch` which did not execute successfully confers no representation authority. An inclusion proof, an equal BlockID, or the chameleon relation alone does not show that a representation was authorized as of H . Serving untrusted remote readers would need authorization evidence tied to an authenticated state root or to proofs of original execution, which this paper does not define.

F. Protected Continuation and Progress

Let $\text{FreshRecv}(w, o, c)$ denote authenticated receipt by an honest owner who has not previously authorized consumption of that instance. Freshness is a history predicate and does not follow from successful receipt alone.

Theorem 4 (Protected continuation): Following FreshRecv , the certified output cannot be reinterpreted or consumed without owner authorization. A certified, publicly executed successor consumes a final source or atomically establishes a funded direct obligation. Source closure, compensation, and representation preserve its established principal effects without successor re-signing or re-execution.

Proof: Lemma 2 fixes ownership and excludes conflicting consumption, and atomic execution couples each missing consumption with its obligation and outputs. Theorem 1 supplies funding, Theorem 2 accounts for closure and late sources, Lemma 1 identifies the executed history, Lemma 4 bounds re-submission, Lemma 6 bounds compensation, and Lemmas 9–11 restrict later changes. Induction over the joint local and public transitions composes these properties. `INSTALL` retains local consumption effects and leaves public assets unchanged. A compensation decision may change Pending, fees, and reserve balances without repeating successor principal, and representation and physical installation change none of them. ■

Local retention cases. The following cases distinguish resource restoration from input unlocking. Elapsed time alone authorizes neither.

Approval state	Consequence under the existing rules
Independent partial approvals without QCs	Capacity may be fragmented without public liabilities. Funded increases can enable completion but do not erase existing locks.
All-honest same-input 2/2 split	Neither candidate can obtain a third signature. Additional capacity does not resolve the conflicting input locks.
Superseded uncertified approval	A successfully executed conflicting QC under the same configuration proves that the old approval could not have formed a QC. The guarded invalidation restores only its residual local debits.
QC-bearing Intent loser	Disjoint-input Intent rejection does not permit invalidation. The unexecuted source retains its signing reservation even after compensation.

Conditional progress. Fix a CAL grant. For member m and Worker w , let $C_{m,w}$ be the locally observed total slice capacity, Available plus Reserved. Let $L_{m,w}$ bound residual reservations outside a designated request set, and let $N_{m,w} \geq 1$ bound its distinct requests that may simultaneously acquire or retain reservations, including the candidate. If each request reserves at most a_{max} CAL, including change, then

$$C_{m,w} \geq L_{m,w} + N_{m,w} a_{\text{max}}$$

excludes a CAL-capacity refusal for that candidate: other reservations occupy at most $L_{m,w} + (N_{m,w} - 1)a_{\text{max}}$. All competing reservations must be counted exactly once, including partial or abandoned approvals, certified but unreleased payments, and payments awaiting local reconciliation.

The condition must hold at the relevant debit steps and slices of a responsive honest quorum. It is a workload condition, not a bound enforced across collectors. Input and Intent compatibility, sufficient fees and other resources, available evidence, and fair delivery and inclusion remain separate requirements. Source execution, economic closure, and representation have distinct progress conditions; none implies a universal completion deadline or timeout-based unlocking.

Lemma 12 (Economic closure does not wait for adaptation): An outstanding direct obligation can be closed by its public compensation decision once its public deadline is reached and the decision is included, provided the original history at its location, the issuer’s reserve, and the consumer’s fee escrow are available. This transition requires no adaptation contribution. Completion of the representation separately requires valid adaptation material, a current canonical revision, and progress of public submission and local physical installation.

Proof: The decision path validates the obligation and the historical slot and performs accounting without an adaptation endpoint. The reserve suffices by Theorem 1, and the admission rule reserves one compensation charge per certified input within the consumer’s escrow. Unavailable adaptation therefore blocks neither the decision nor a later repayment. The threshold service needs three valid contributions from the four fixed holders; three responding honest holders suffice, and a Byzantine holder may also supply a valid contribution. Shared execution resources and public ordering still govern latency, so the statement separates protocol dependencies and gives no completion-time bound. ■

Availability and guarantor risk. Once a consumer exposes the source QC, at least two honest source signers retain its

submission material (Lemma 4). Fair delivery and inclusion support resubmission of an executable source but cannot overcome a permanent Intent conflict; a due obligation can instead close without adaptation (Lemma 12). At every prefix,

$$0 \leq S_g = \sum_{c \in C_g} (p_c - \text{Rec}_c) \leq Q_g \leq B_g.$$

For a fixed grant without further funding, this bounds net unrepaid compensation, not lifetime gross Paid. An unexecuted certified source keeps $r_{m,c} = a_c$ at its honest signers, so a slice can stop issuing while reserve principal remains. A funded top-up increases shares without clearing reservations or input conflicts and exposes the new authorization to the same risk. Permanently unexecuted sources fall outside Theorem 3. The open service in Section III-E accepts adversarial reserve loss and retained capacity; FUEL fees do not insure either, and no CAL–FUEL exchange value is assumed. Supplement S3 gives the two-round ledger and its fixed-signer capacity bound.

VI. IMPLEMENTATION AND EVALUATION

Builds and measurement. The current-build series is based on `f856c7b`: three 100-hop runs per continuation mode and three runs of the representative paused-adaptation treatment P. All nine runs share the same node build. Additional timing probes are confined to `payctl` and are enabled in all six chain runs, without changes to production node paths. The complete N/R/C/P matrix on `721800c` remains archived; historical-reader measurements retain `721800c`, and the injected-latency study retains `9879f64`. Chain summaries use medians and ranges of three run totals; mixed-load summaries use medians of three run-level statistics.

Setup and measurement. Experiments use one Mac Studio M4 Max with 16 CPU cores, 64 GiB RAM, macOS 26.6.2, and Go 1.27.1. The project overlay of CometBFT v0.38.26 implements the identity and installation rules. Network trials use synchronous `bbolt` member commits before signing; committee application state, `BlockStore`, and gateways are disk-backed. Members use `GOMAXPROCS=16` and `GOGC=200`; commit, flush, and gossip settings are 250, 10, and 10 ms. Network trials copy the same unrun genesis and keys, reset genesis time, and use sufficient fixed funding without top-up.

Certified receipt runs from first actual send to recipient verification and atomic local recording. Public and member observations include block following and driver polling; the chain experiment separately records application-commit completion. Unless stated otherwise, summaries take the median of three run-level statistics, not pooled transaction quantiles; ranges describe runs, not confidence intervals. The timing trials do not measure restart recovery, sustainable throughput, or minimum capital. The persistent restart checks below are separate functional regressions.

A. Continuous Spending Without Per-Hop Settlement

Nine services support three independent NoSync wallets rotating 100 CAL through online 100-hop chains, using synchronous member commits and user-funded final FUEL. The

three pairs run in fast/wait, wait/fast, fast/wait order after a uniform three-second warm-up. Fast mode proceeds after certified receipt; wait mode also requires the driver’s observation of wallet-verified public execution between hops. Both measure $\text{Sent}_1 \rightarrow \text{Ready}_{100}$, including construction of hops 2–100 and retries, but excluding first-payment construction. Wait mode adds 99 intermediate waits, not the final payment’s public confirmation.

Median chain times are 4.707 s and 64.479 s, respectively, a 13.70-fold ratio of medians; run ranges are 4.700–4.726 s and 64.387–64.670 s. Fast uses 300 attempts for 300 payments, whereas wait uses 414. All 297 fast successors use the preceding TXCer and are first sent before the earliest parent application-commit completion across the four replicas, with a minimum lead of 183.439 ms. Actual child-before-source obligations number 2, 4, and 1 in the three fast runs; early sending alone is not counted as out-of-order execution. All 600 payments close and pass the accounting, input, replica, fee, and outbox audits. Fast-mode all-payment member closure has a median of 5.526 s from the first send. Last-payment public observation has a median of 5.477 s. Each chain spends 9,400 FUEL: 8,400 in rewards and 1,000 burned. For the fixed transaction shape, the TXCer is 779 B and requests are 1,979 B initially and 2,766 B thereafter.

Client timing. For target height h , `BlockData` obtains the signed header at $h + 1$ before fetching the block and results at h . The RPC can supply the latest-height seen commit without waiting for $h + 2$. From application-commit completion at the fetch source, `committee0`, commit-to-complete-fetch includes successor evidence, replica progress, polling, and retrieval, not pure polling or query overhead. Its wait-mode P50/P95 are 285.884/314.509 ms. Separate P95s for post-fetch verification, preparation/local recording, and post-transaction callback-to-driver observation are 0.236, 1.343, and 4.713 ms. The callback does not await a durability flush. Supplement S4 specifies checkpoints, clocks, matching, and aggregation. The chain ratio compares end-to-end modes, including retries and evidence observation; negative fast observation-to-build differences indicate overlap.

B. Source Resolution Under Representative Mixed Load

The fourteen-service P treatment schedules 7,000 ordinary payments at 100 payments/s over 70 s, alongside eight independent cross-organization parent–child pairs, using authorized organization-funded FUEL. Four committee nodes and two organizations with four members and one gateway each give fourteen services. Ordinary wallets use NoSync; scenario wallets commit synchronously. The driver limits fast requests to 64 and outstanding payments to 256. Pairs start ten seconds after driver launch, one second apart, and scenario wallets follow blocks for at least 55 s. Parent publication and `INSTALL` are withheld, automatic source resubmission is disabled, and all adaptation endpoints are paused; consensus and compensation decisions remain active. Each late source is released after its compensation is observed. Ordinary traffic spans the injected fault and recovery intervals.

All 21,048 payments across the three runs close, including 21,000 ordinary payments and 24 parent–child pairs. Each run

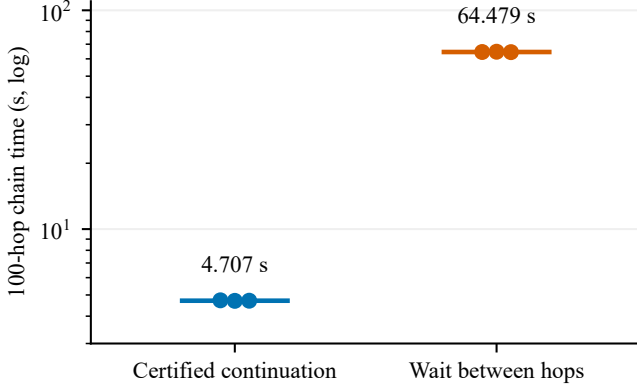


Fig. 4. Current-build 100-hop chains on f856c7b, including client timing probes: three runs per mode, synchronous members and NoSync wallets. Dots are run totals; bars mark medians. Only wait mode requires public observation between hops.

pays and recovers 800 CAL. Before adaptation resumes, all four committee replicas report all eight targets *Recovered*, with their representation *Committed* and *Materialized* flags both false. These 96 replica-target checks establish compensation and repayment before adaptation resumption in the measured executions. Within each run, committee state hashes agree at a common height, and CAL/FUEL, input, fee, reservation, and outbox audits pass. Per-run fees are 659,544 FUEL: 589,384 in rewards and 70,160 burned.

Table III reports ordinary receipt latency together with pre-send backpressure and drain. Median run-level receipt P50/P95 are 54.538/147.549 ms; public and member-observation P95s are 2.187 and 2.255 s. All three runs reach the 256-outstanding limit. Dispatch P95 reaches 780.727 ms and scheduled-to-receipt P95 reaches 851.742 ms; the latter is computed per payment, not by adding quantiles. Actual send spans are 70.080–70.771 s, followed by 0.955–1.115 s of ordinary-payment drain. This representative P series demonstrates economic closure during ordinary traffic with adaptation unavailable; it is not a current-build comparison of interference across N/R/C/P.

C. Historical Reading: Equivalent Answers and Explicit Costs

We evaluate the exception-review workflow in Section IV-H: whether revised records preserve economic answers and the original complete block identity, and what local reading, storage, and maintenance they cost. The paused-adaptation runs above separately test that economic closure proceeds before representation.

The timed historical-reader series retains build 721800c; the same-consumer functional extension uses the current build. The reader is an honest local replica that has verified and executed prefix H , querying original coordinates (h, j, k) within one application read transaction V_H . A loads original history and uses the permanent decision index; B uses existing Canonical revisions. Both consult the same permanent decisions and obligations, returning identical input identity, TxID, coordinates, amount, historical funding/debit, decision/height,

and obligation state at H . Authorization comes from successful execution binding exact revisions, not a CH equation or matching BlockID. This is not remote-server authentication.

Body compatibility versus economic answers. An extended archival-consumer test compares A and B with M, a plain serialized materialized funding row keyed by the same original identity and coordinates. M caches immutable funding and decision fields and reads current obligation status from V_H ; it does not adapt any commitment. All three return equal economic answers before and after real repayment. B additionally verifies its authorized body against the actual saved original commit. Rebuilding that changed body with initial part openings preserves the header hash but changes the part-set header and is rejected by the same commit check. After repayment, the authorized body and M’s cached rows remain unchanged while the obligation becomes *Recovered*. Table IV records these functional results. M is a test-only cached view, not a third timed backend.

Fixtures use real signatures, successful ABCI execution, disk-backed bbolt/GoLevelDB, and frozen BlockStores. A has its own original BlockStore, without B’s backup-layout cost. Point queries load a block and decode the selected transaction; A’s block query scans its range index once and builds a timed map. Neither strategy receives a precomputed decision map. Pages are warm, with no decoded-block or result cache; GOMAXPROCS=16, GOGC=100.

Three fresh processes each test four (n, m) fixtures, with $n \in \{32, 256\}$ payments and $m \in \{1, 8\}$ compensated slots per block. After 20 warm-ups per strategy, 300 interleaved queries per strategy for compensated points, ordinary points, and whole blocks yield 21,600 timed queries. Timing includes read-transaction boundaries, loading/decoding, record association, and result construction. All timed reads use represented, materialized $H = 4$; other stages and subsequent real repayments are functional-equivalence checks, not extra timing samples.

Canonical reduces compensated-point P50 by 7.44–12.32% (12.6–123.8 μ s); whole-block differences are 1.49–3.88% (Table V). In (256, 8) run 1, A performs 12 BlockStore Gets returning 622,183 B; B performs none but reads 825,526 B from application state. Mean per-query heap allocation is approximately 4.95 versus 2.24 MB, while record-association medians are 11.50 versus 10.83 μ s. This indicates a layout/reconstruction benefit, not fewer total bytes or cryptographic acceleration. Decoded-object caching could change that tradeoff.

Representation adds 0.28–2.19 MiB of net logical KV per repaired block, for revisions, tasks, and backup/history records, net of deleted Pending. The (256, 8) 5,056-B batch is already included. Local sequential adaptation, assembly, Finalize, synchronous application Commit, and materialization total 43–214 ms. It excludes network/consensus waits, public-block saving, and commit-signature construction. Common compensation costs are reported separately. These figures characterize incremental repair on a CH-enabled build; they do not isolate the steady-state cost of CH encoding against a non-CH build.

Equivalence checks cover compensation before representation, representation before materialization, and repayment

TABLE III

ORDINARY-PAYMENT MEASUREMENTS FOR REPRESENTATIVE P RUNS ON P856C7B. EACH RUN CONTAINS 7,000 ORDINARY PAYMENTS PLUS EIGHT PARENT-CHILD PAIRS. RECEIPT STARTS AT ACTUAL SEND; DISPATCH SPANS SCHEDULED TO ACTUAL SEND; SCHEDULED-TO-RECEIPT IS CALCULATED DIRECTLY PER PAYMENT. DRAIN STARTS AT THE LAST ORDINARY SEND. THE FINAL ROW TAKES THE MEDIAN OF EACH COLUMN'S THREE RUN-LEVEL ENTRIES.

Run	Receipt P50 (ms)	Receipt P95 (ms)	Dispatch P95 (ms)	Scheduled-to-receipt P95 (ms)	Send span (s)	Drain (s)
P-1	54.538	156.680	90.335	189.591	70.080	1.090
P-2	54.007	147.549	245.315	326.828	70.235	0.955
P-3	55.001	141.424	780.727	851.742	70.771	1.115
Median	54.538	147.549	245.315	326.828	70.235	1.090

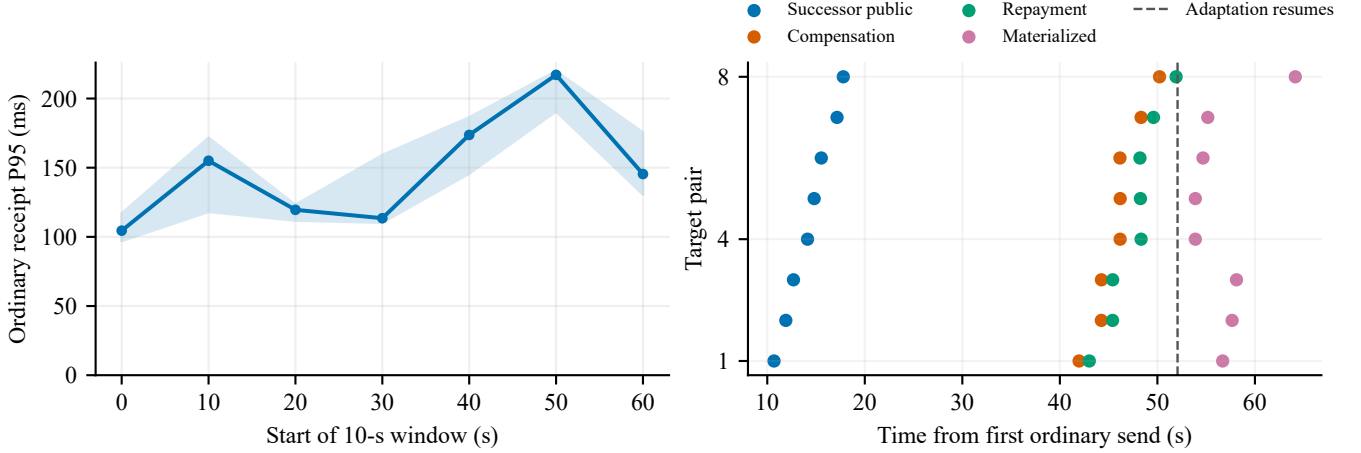


Fig. 5. Current-build P observations. Left: ordinary-receipt P95 in the first seven ten-second windows; line and band show three-run medians and minima-maxima. Right: eight targets in P-1, with compensation and repayment preceding adaptation resumption. Public events are observations, not internal commit timestamps.

TABLE IV

FUNCTIONAL COMPARISON FOR LOCAL ARCHIVAL REVIEW. A: ORIGINAL BODY PLUS DECISION INDEX; M: ORDINARY CACHED FUNDING ROWS; B: C3. ALL USE THE SAME EXECUTED PREFIX AND AUTHORIZATION RECORDS. THE REVISED-BODY ROW IS C3'S REPRESENTATION CONTRACT.

Capability	A	M	B
Equal economic answers at H	Yes	Yes	Yes
Retain original coordinates and ID	Yes	Yes	Yes
Original body validates original commit	Yes	Yes	Yes
Reserve-debit reference inside a revised body that validates the same commit	No	No	Yes
Consult public authorization and state	Yes	Yes	Yes

preserving the historical debit with state Recovered. Invalid coordinates, cross-prefix decisions, and wrong debit/amount are rejected; undecodable revisions fail rather than fall back. These checks do not establish detection of arbitrary decodable corruption. Canonical offers authorized funding representation at stable historical coordinates, with measured maintenance costs, rather than an economically necessary or universally faster alternative to indexed history.

D. Injected Latency and Single-Fault Runs

On 9879f64, four conditions run three times each in alternating order. H uses the injection proxies without added delay; L adds 25 ms per direction to member RPC and committee P2P traffic. L-M and L-C add that delay and suspend member 3 or committee validator 3, respectively, from 15 to 45 s. Each run sends 100 warm-up payments, 1,200 ordinary payments at 20/s, and two 10-hop chains starting inside the fault window. The actual health-RPC RTT rises from 1.17 to 52.20 ms; both RPC and P2P mean holds are about 25.2 ms per direction. The supplement specifies the FIFO injection, fixed resources and per-run results.

All 15,840 payments close, with equal committee states, empty outboxes and zero public gap. No compensation is needed. Table VI separates receipt, public observation and observation of all four members. With one member suspended, new QCs use the other three members and public completion continues, while the absent member's observation creates a peak of 630–641 outstanding tasks. With a validator suspended, receipt P95 remains 108–111 ms and 400–510 payments reach public observation inside each fault window, but public P95 rises to 9.5–12.6 s. Retained commit certificates verify five paused-validator round-0 proposer opportunities per run; each commits in round 1 without that validator's signature. Both chains finish certification within each fault

TABLE V

HISTORICAL READING AND REPRESENTATION COSTS. A IS ORIGINAL HISTORY PLUS DECISION INDEX; B IS CANONICAL. POINT READS TARGET COMPENSATED SLOTS. QUERY ENTRIES ARE MEDIAN OF THREE RUN-LEVEL P50S; COST ENTRIES ARE MEDIAN OF THREE TOTALS. EXTRA KV EXCLUDES COMMON HISTORY, DECISIONS, AND APPLICATION COMMIT METADATA.

(n, m)	Point (μ s)		Whole block (μ s)		Extra net KV (MiB/block)	Local work (ms/block)
	A	B	A	B		
(32, 1)	162.54	148.38	1,028.29	1,005.79	0.279	43.32
(32, 8)	169.21	156.62	1,127.46	1,083.71	0.314	94.46
(256, 1)	1,005.21	881.38	8,441.58	8,316.00	2.153	65.32
(256, 8)	1,006.25	887.25	8,554.17	8,407.12	2.189	214.40

TABLE VI

INJECTED LATENCY AND SINGLE FAULTS: MEDIAN OF THREE RUN-LEVEL STATISTICS OVER ALL 1,200 ORDINARY PAYMENTS PER RUN; WARM-UP AND CHAINS ARE EXCLUDED. PUBLIC AND MEMBER COLUMNS ARE SEND-TO-OBSERVATION P95; THE MEMBER COLUMN WAITS FOR ALL FOUR MEMBERS.

Condition	Receipt P50/P95 (ms)	Public (s)	Members (s)
H	41.56/83.20	1.28	1.36
L	81.91/118.22	1.32	1.41
L-M	77.71/112.85	1.30	29.00
L-C	77.41/109.32	10.02	10.04

window, and every run drains within 1.25 s of its last ordinary send. These are single-host injected-delay results, separate from the fast/wait comparison.

E. Boundary and Composition Checks

Deterministic tests on 9879f64 use actual member approvals, ABCI execution, and authenticated follower updates. They reproduce no-QC capacity fragmentation, a same-input 2/2 split, and two Intent-conflict compensation rounds with complete CAL/FUEL accounting. An external top-up resolves the first case but not the second. Real threshold adaptation covers both representation/repayment orders; retained-state restart reproduces each original command history. A commit-before-signature interruption preserves locks and reservations and permits an idempotent retry. The supplement gives the per-prefix ledgers, negative-entry tests, and restart assertions. The current-build proof-to-code map identifies the entry points, atomic guards, and existing regressions supporting each property. An additional raw-byte ABCI test rejects unrelated or forged QCs and forged owner signatures at CheckTx, ProcessProposal, and FinalizeBlock without business-state changes.

Archived evidence. The supplement retains earlier continuation, throughput, capital, organization-path, and availability experiments under their original builds and configurations. They provide historical context, not measurements of f856c7b; cross-version differences are not attributed to individual changes.

VII. CONCLUSION

Next separates continuation, economic closure and historical record. On one host with synchronous member commits,

certified continuation reaches the last receipt of a 100-hop chain in 4.707 s, against 64.479 s when each hop waits for public execution (retries included). Reserve-backed signing capacity covers certificate liability, original signers can re-submit exposed sources, and a due compensation decision closes an obligation that a late source repays, even with adaptation paused. Decision-authorized funding-reference repair then records the historical reserve debit in the existing block record, without changing its complete identity or repeating its economic effects. Each path advances on its own evidence while preserving the authorization and outputs of completed payments.

ACKNOWLEDGMENTS

ChatGPT and Claude assisted with drafting and revising the abstract, main-text sections, supplementary explanations, and Chinese reading edition. Codex assisted with source-grounded review, implementation and test scaffolding, revision, LaTeX assembly, and plotting scripts. Experimental plots were generated from recorded measurements.

REFERENCES

- [1] M. Baudet, G. Danezis, and A. Sonnino, “FastPay: High-Performance Byzantine Fault Tolerant Settlement,” in *Proceedings of the ACM Conference on Advances in Financial Technologies*, 2020, pp. 163–177.
- [2] M. Baudet, A. Sonnino, M. Kelkar, and G. Danezis, “Zef: Low-latency, Scalable, Private Payments,” in *Proceedings of the Workshop on Privacy in the Electronic Society*, 2023, pp. 1–16.
- [3] S. Blackshear, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, X. Li, M. Logan, A. Menon, T. Nowacki, A. Sonnino, B. Williams, and L. Zhang, “Sui Lutris: A Blockchain Combining Broadcast and Consensus,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 2606–2620.
- [4] L. Aumayr, P. Moreno-Sanchez, A. Kate, and M. Maffei, “Blitz: Secure Multi-Hop Payments Without Two-Phase Commits,” in *30th USENIX Security Symposium*, 2021, pp. 4043–4060. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/aumayr>
- [5] —, “Breaking and Fixing Virtual Channels: Domino Attack and Donner,” in *Network and Distributed System Security Symposium*, 2023.
- [6] A. Shamis, P. Pietzuch, B. Canakci, M. Castro, C. Fournet, E. Ashton, A. Chamayou, S. Clebsch, A. Delignat-Lavaud, M. Kerner, J. Maffre, O. Vrousseau, C. M. Wintersteiger, M. Costa, and M. Russinovich, “IA-CCF: Individual Accountability for Permissioned Ledgers,” in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, 2022, pp. 467–491. [Online]. Available: <https://www.usenix.org/conference/nsdi22/presentation/shamis>
- [7] S. A. K. Thyagarajan, A. Bhat, B. Magri, D. Tschudi, and A. Kate, “Reparo: Publicly Verifiable Layer to Repair Blockchains,” in *Financial Cryptography and Data Security*, 2021, pp. 37–56. [Online]. Available: <https://arxiv.org/abs/2001.00486>

- [8] CometBFT Contributors, “CometBFT Core Data Structures: BlockID and Commit,” Protocol specification, version 0.38, accessed October 4, 2026. [Online]. Available: https://docs.cometbft.com/v0.38/spec/core/data_structures
- [9] S. Sridhar, A. Sonnino, and L. Kokoris-Kogias, “Stingray: Fast Concurrent Transactions Without Consensus,” 2025, arXiv:2501.06531, version 1. [Online]. Available: <https://arxiv.org/abs/2501.06531>
- [10] S. Mazumdar and S. Ruj, “CryptoMaze: Privacy-Preserving Splitting of Off-Chain Payments,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 2, pp. 1060–1073, 2023.
- [11] V. Mavroudis, K. Wüst, A. Dhar, K. Kostiaenen, and S. Čapkun, “Snappy: Fast On-Chain Payments with Practical Collaterals,” in *Network and Distributed System Security Symposium*, 2020. [Online]. Available: <https://arxiv.org/abs/2001.01278>
- [12] A. Madhusudan, M. Sedaghat, S. Tiwari, K. Cong, and B. Preneel, “Reusable, instant and private payment guarantees for cryptocurrencies,” in *Information Security and Privacy (ACISP)*, 2023, pp. 580–605.
- [13] Z. Liu, Y. Xiang, J. Shi, P. Gao, H. Wang, X. Xiao, B. Wen, Q. Li, and Y.-C. Hu, “Make Web3.0 Connected,” *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 2965–2981, 2022.
- [14] G. Ateniese, B. Magri, D. Venturi, and E. Andrade, “Redactable Blockchain—or—Rewriting History in Bitcoin and Friends,” in *IEEE European Symposium on Security and Privacy*, 2017, pp. 111–126. [Online]. Available: <https://eprint.iacr.org/2016/757>
- [15] D. Zhang, J. Le, X. Lei, T. Xiang, and X. Liao, “Secure Redactable Blockchain With Dynamic Support,” *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 2, pp. 717–731, 2024.
- [16] X. Li, J. Xu, L. Yin, Y. Lu, Q. Tang, and Z. Zhang, “Escaping From Consensus: Instantly Redactable Blockchain Protocols in Permissionless Setting,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 5, pp. 3699–3715, 2023.
- [17] K. Y. Chan, L. Chen, Y. Tian, and T. H. Yuen, “Reconstructing Chameleon Hash: Full Security and the Multi-Party Setting,” in *Proceedings of the ACM Asia Conference on Computer and Communications Security*, 2024, pp. 1076–1091.
- [18] V. Shoup, “Practical Threshold Signatures,” in *Advances in Cryptology—EUROCRYPT*, 2000, pp. 207–220. [Online]. Available: <https://www.iacr.org/archive/eurocrypt2000/1807/18070209-new.pdf>
- [19] R. Matzutt, V. Ahlrichs, J. Pennekamp, R. Karwacik, and K. Wehrle, “A Moderation Framework for the Swift and Transparent Removal of Illicit Blockchain Content,” in *IEEE International Conference on Blockchain and Cryptocurrency*, 2022.
- [20] X. Chen and Y. Gao, “CDEdit: A Highly Applicable Redactable Blockchain with Controllable Editing Privilege and Diversified Editing Types,” 2022, arXiv:2205.07054. [Online]. Available: <https://arxiv.org/abs/2205.07054>
- [21] Cloudflare, “CIRCL v1.6.5: Threshold RSA implementation,” Software, tss/rsa package, accessed October 3, 2026. [Online]. Available: <https://github.com/cloudflare/circl/tree/v1.6.5/tss/rsa>
- [22] E. Buchman, J. Kwon, and Z. Milosevic, “The latest gossip on BFT consensus,” 2019, arXiv:1807.04938, version 3. [Online]. Available: <https://arxiv.org/abs/1807.04938v3>